injection in chapter 7, but for the time being, it is enough to know that you need to add the bold line of code in the next listing to the `Program` class.

Listing 2.17 Registering `IMiddleware` as a service

```
builder.Services.AddRazorPages();
builder.Services.AddScoped<IpAddressMiddleware>();
```

So why are there two different ways to create middleware classes, and which one should you use? Well, the conventions-based approach requires that you learn the specific conventions and remember them. There is no compile-time checking to ensure your middleware implements the conventions correctly. This approach is known as *weakly typed*. Typically, the first time you discover that you forgot to name your method `Invoke` or `InvokeAsync` or that the first parameter should be an `HttpContext` will be when you try to run the application, and it falls over. If you are anything like me, you will often find that you have to refer back to the documentation to remind yourself of the convention details.

The second approach results in strongly typed middleware because you have to implement the members of the `IMiddleware` interface; otherwise, the compiler complains, and your application won't even build. So the `IMiddleware` approach is less error prone and potentially quicker to implement, although you do have to take the extra step of registering the middleware with the service container.

There is another difference between the two approaches. I mentioned earlier that convention-based middleware is instantiated as a singleton when the pipeline is first built. `IMiddleware` components are instantiated for each request by a component that implements the `IMiddlewareFactory` interface, and this difference has ramifications for services the middleware depends on, based on their